

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Department:	Information Technology Artificial Intelligence and Data Science
Programme:	B. Tech
Name	1. Janessa J. 2. Lochana Sri R S 3. Gayatheri S.
Course Code & Course Name	CSA1202 – Computer Architecture
Academic Year / Batch	2024 & 2025
Faculty Name	Dr. Vasavi J.
Assignment Title	Precision Agriculture using Computer Architecture
Date of Issue	01. 09. 2026
Date of Submission	01. 09. 2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. CO3: Design processor organization by constructing pipelined data path structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT).
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse
SDG Mapping (SDG 1-17)	SDG 2 - Zero Hunger
Industry / Societal Relevance	Relevant to precision agriculture and smart farming by demonstrating sensor-based irrigation decision-making and efficient processor execution, with potential benefits for water conservation and agricultural productivity.

PRECISION AGRICULTURE USING COMPUTER ARCHITECTURE

Smart Irrigation System Using a Pipelined Processor

1. Problem Understanding and Formulation

1.1 Problem Statement

The proposed system is a Smart Irrigation System that processes soil-moisture, temperature, and humidity sensor values and determines whether irrigation is required. The system is implemented as an educational computer architecture simulator that demonstrates how sensor data can be processed through arithmetic units and a 5-stage pipelined processor.

The implementation accepts three environmental parameters:

- Soil Moisture (%)
- Temperature (°C)
- Relative Humidity (%)

The input values are processed to calculate the water requirement and determine the irrigation status. The processor simulation demonstrates the execution of this processing using:

- Integer arithmetic
- Booth's multiplication algorithm
- Restoring division algorithm
- IEEE-754 single-precision representation
- 5-stage instruction pipelining
- Data hazard detection
- Forwarding
- Load-use interlocks
- Structural hazard simulation
- Dynamic branch prediction
- Performance comparison between optimized and unoptimized execution

The final output is an irrigation decision and calculated water-flow rate in **L/min**.

1.2 Objective

The main objective of the implementation is to demonstrate how computer architecture concepts can be applied to a smart agriculture application.

The specific objectives are:

1. Accept soil moisture, temperature, and humidity values as sensor inputs.
2. Represent sensor values in binary and IEEE-754 formats.
3. Calculate irrigation demand using the implemented irrigation model.
4. Demonstrate hardware arithmetic using Booth's multiplication and Restoring division.
5. Process the irrigation operation using an 11-instruction sequence.
6. Simulate execution through a 5-stage RISC pipeline.
7. Detect and resolve RAW data hazards using forwarding.
8. Handle Load-Use hazards using a one-cycle stall.
9. Simulate structural hazards caused by memory-bus contention.
10. Demonstrate branch prediction using a 2-bit saturating-counter FSM.
11. Compare optimized and unoptimized processor execution.
12. Calculate CPI, throughput, latency, speedup, stall reduction, and estimated energy savings.
13. Provide the final irrigation actuation status and flow rate.

2. Requirements and Constraints

2.1 Functional Requirements

The implemented system provides the following functions:

- Interactive sensor input.
- Soil moisture input from 0–100%.
- Temperature input from 0–50°C.
- Humidity input from 0–100%.
- Target soil moisture configuration.
- Base water requirement configuration.
- Irrigation demand calculation.
- Irrigation status classification.
- Water-flow calculation.
- Binary representation of sensor values.
- IEEE-754 analysis.
- Booth multiplication simulation.
- Restoring division simulation.
- Integer ALU operations.
- 5-stage pipeline simulation.
- Hazard detection and visualization.

- Forwarding visualization.
- Branch prediction simulation.
- Performance evaluation.
- JSON and CSV result export.

2.2 Technical Constraints

The implementation is a software-based educational simulator, rather than a physical agricultural control device.

The sensor values are simulated through user inputs. The addresses such as `0x2000`, `0x2004`, and `0x2008` represent simulated memory-mapped ADC registers.

The irrigation output is also simulated through the logical solenoid-valve register at `0x3000`.

Therefore, the current implementation does not directly communicate with physical sensors, ADC hardware, pumps, or valves.

3. Input Data and Dataset

3.1 Dataset Used

No external dataset is used in the implementation.

The system uses:

1. **Interactive sensor inputs** entered through the application.
2. **Default sensor values** for simulation.
3. A **hardcoded 15-cycle synthetic sensor sequence** for evaluating branch prediction.

The default sensor configuration in the implementation is:

Parameter	Default Value
Soil Moisture	35%
Temperature	31°C
Relative Humidity	58%
Target Moisture	60%
Base Water Requirement	8 L/min

The branch prediction module separately contains a predefined 15-cycle sequence representing changing soil and temperature conditions throughout a simulated day.

Therefore, the project should **not claim to use a real agricultural dataset or IoT sensor dataset**.

4. Application of Computer Architecture Concepts

The implementation applies the following concepts from computer architecture.

4.1 Integer Arithmetic

The system demonstrates addition, subtraction, multiplication, and division through an interactive ALU module.

4.2 Booth's Multiplication

Booth's signed multiplication algorithm is implemented using two's-complement representation.

The algorithm checks the pair:

$Q_0 Q_{-1}$

and performs:

$Q_0 Q_{-1}$	Operation
00	No arithmetic operation
01	$A = A + M$
10	$A = A - M$
11	No arithmetic operation

After the operation, an **Arithmetic Right Shift** is performed on $[A, Q, Q_{-1}]$.

The implementation supports configurable bit widths, with the interactive interface using a default **5-bit** configuration.

4.3 Restoring Division

The implementation also provides a Restoring Division simulator.

The implemented sequence is:

1. Initialize accumulator $A = 0$.
2. Load dividend into Q .
3. Load divisor into M .
4. Shift $[A, Q]$ left.
5. Perform $A = A - M$.
6. If the result is negative, restore A using $A = A + M$ and set $Q_0 = 0$.
7. Otherwise retain the subtraction result and set $Q_0 = 1$.
8. Repeat for the selected number of bits.
9. Obtain quotient from Q and remainder from A .

The interface allows the dividend, divisor, and bit width to be changed interactively.

5. IEEE-754 Floating-Point Representation

The implementation contains an IEEE-754 single-precision analyzer.

A floating-point value is represented using:

- **1 sign bit**
- **8 exponent bits**
- **23 mantissa/fraction bits**

The implementation extracts the actual 32-bit representation using `Float32Array` and `Uint32Array`.

The displayed representation includes:

- Sign bit
- Exponent bits
- Biased exponent
- Unbiased exponent
- Mantissa
- 32-bit binary representation

- Hexadecimal representation
- Reconstructed floating-point value
- Floating-point classification

The mathematical reconstruction displayed by the application is:

$$\text{Value} = (-1)^s \times (1 + \text{Mantissa}) \times 2^{(\text{Exponent} - 127)}$$

for normalized values.

6. Fixed-Point Representation

The implementation also contains a **Q8.8 fixed-point conversion function**.

In this representation:

- 8 bits represent the integer portion.
- 8 bits represent the fractional portion.
- Total representation = 16 bits.

The input is multiplied by 256 and rounded before being converted to a 16-bit two's-complement representation.

The system displays:

- Raw fixed-point value
- Binary representation
- Hexadecimal representation
- Reconstructed decimal value

7. Proposed System Architecture

The implemented system follows the following processing flow:

Sensor Input

↓

ADC / Memory-Mapped Sensor Representation

↓

Binary / IEEE-754 Conversion

↓

Irrigation Demand Calculation

↓

Instruction Generation

↓

5-Stage RISC Pipeline

↓

Hazard Detection + Forwarding

↓

Branch Prediction

↓

Memory / Actuation

↓

Irrigation Decision + Performance Evaluation

The application provides separate views for these processing stages, allowing the user to inspect both the agricultural calculation and the underlying processor behavior

8. Sensor Input Module

The Sensor Input module provides interactive controls for:

Soil Moisture

- Range: 0–100%
- Simulated ADC address: 0x2000
- ADC Channel: 0

Temperature

- Range: 0–50°C
- Simulated ADC address: 0x2004
- ADC Channel: 1

Relative Humidity

- Range: 0–100%
- Simulated ADC address: 0x2008
- ADC Channel: 2

The application provides both sliders and numeric input fields.

Each sensor value is additionally displayed in an 8-bit binary representation and an IEEE-754 hexadecimal representation.

9. Irrigation Decision Algorithm

The irrigation calculation is implemented in the `calculateIrrigationDemand()` function.

The calculation consists of four components.

9.1 Soil Moisture Deficit

The target moisture is compared with the current soil moisture.

$$\text{MoistureDeficit} = \max(0, \text{TargetMoisture} - \text{SoilMoisture})$$

The implemented deficit coefficient is:

0.75 \text{ L/min per 1\% deficit}

Therefore:

$$\text{DeficitComponent} = \text{MoistureDeficit} \times 0.75$$

9.2 Temperature Component

The implementation uses **20°C as the temperature baseline**.

$$TemperatureExcess = \max(0, Temperature - 20)$$

The temperature coefficient is:

$$0.60 \text{ L/min per } ^\circ\text{C}$$

Therefore:

$$TemperatureComponent = TemperatureExcess \times 0.60$$

9.3 Humidity Component

Higher humidity reduces the calculated irrigation demand.

The implemented calculation is:

$$HumidityComponent = 100Humidity \times 15$$

This value is subtracted from the total demand.

9.4 Total Water Demand

The complete implemented equation is:

$$WaterDemand = BaseRequirement + DeficitComponent + TemperatureComponent - HumidityComponent$$

The final flow rate is restricted to a range of:

$$2 \leq FlowRate \leq 65 \text{ L/min}$$

when irrigation is required.

10. Irrigation Decision Conditions

The implementation classifies the result into four states.

1. Over-saturated

If:

$SoilMoisture > 75\%$

the system produces:

Over-saturated – Drainage Alert

and sets irrigation flow to zero.

2. Critical Irrigation

If:

$SoilMoisture < 35\%$

OR

$SoilMoisture < 50\%$ and $Temperature > 32^{\circ}C$

the system produces:

Critical – Immediate Irrigation

and ensures that the raw demand is at least 15 L/min.

3. Moderate Irrigation

If:

$SoilMoisture < TargetMoisture$

and calculated demand is greater than 5 L/min, the system produces:

Moderate Irrigation Required

4. Optimal

Otherwise:

Optimal – No Irrigation

11. Example Using the Default Input

For the default values:

- Soil Moisture = 35%
- Temperature = 31°C
- Humidity = 58%
- Target Moisture = 60%
- Base Requirement = 8 L/min

Soil moisture deficit

$$60 - 35 = 25\% \quad 25 \times 0.75 = 18.75$$

Temperature component

$$31 - 20 = 11^\circ\text{C} \quad 11 \times 0.60 = 6.60$$

Humidity component

$$(58/100) \times 15 = 8.70$$

Total demand

$$8 + 18.75 + 6.60 - 8.70 = 24.65 \text{ L/min}$$

Therefore, the implemented system produces:

Irrigation Required = YES

Status = Moderate Irrigation Required

Flow Rate = 24.65 L/min

This example can be directly used in the report as the main test case.

12. Instruction Set Used in the Processor

The irrigation processing is represented using an **11-instruction sequence**.

Instruction	Operation	Purpose
I1	LOAD	Read soil moisture
I2	LOAD	Read temperature

Instruction	Operation	Purpose
I3	LOAD	Read humidity
I4	SUB	Calculate soil moisture deficit
I5	MUL	Calculate temperature/evaporation factor
I6	ADD	Accumulate water requirement
I7	SUB	Apply humidity reduction
I8	CMP	Compare calculated demand
I9	BGT	Decide irrigation branch
I10	STORE	Write flow command to valve register
I11	STORE	Write telemetry information

The simulated addresses are:

- 0x2000 → Soil Moisture
- 0x2004 → Temperature
- 0x2008 → Humidity
- 0x3000 → Solenoid Valve output
- 0x3004 → Telemetry log

13. Five-Stage Pipelined Processor

The implementation models a standard five-stage pipeline:

IF – Instruction Fetch

The processor fetches the instruction using the Program Counter.

The implemented stage includes:

- Program Counter
- Instruction Memory
- PC + 4 adder

ID – Instruction Decode

The instruction is decoded and source registers are identified.

The simulated stage includes:

- Register file
- Immediate sign extension
- Hazard detection unit

EX – Execute

Arithmetic and branch operations are executed.

The simulated stage contains:

- 32-bit ALU
- Booth multiplier
- Branch target calculation
- Forwarding multiplexers

MEM – Memory Access

Memory operations are performed.

The implementation represents:

- Data memory
- Sensor ADC memory-mapped I/O
- Solenoid valve I/O

WB – Write Back

The result is written back into the register file.

14. Data Structures Used

The implementation uses TypeScript interfaces and arrays to represent the processor state.

Sensor Data

Stores:

- Soil moisture
- Temperature
- Humidity
- Target moisture
- Base water requirement

Irrigation Decision

Stores:

- Irrigation requirement
- Flow rate
- Moisture deficit
- Temperature factor
- Humidity factor
- Raw score
- Status
- Formula breakdown

Instruction Def

Stores:

- Instruction ID
- Program counter
- Mnemonic
- Operands
- Instruction type
- Source registers
- Destination register
- Description
- Branch information

Instruction Execution Record

Stores the pipeline history of each instruction.

Arrays

Arrays are used to store:

- Instruction sequences
- Pipeline progress
- Hazard information
- Forwarding events
- Branch history
- Arithmetic algorithm steps

Record/Map Structures

JavaScript/TypeScript record structures are used for:

- Register values
- Memory values

- Pipeline stage history
- Branch-state labels

15. Data Hazard Detection

The implementation detects **Read After Write (RAW)** dependencies.

For example:

I6: ADD R6, R4, R5

depends on the results generated by earlier instructions.

Instead of allowing the processor to wait unnecessarily for every result to reach WB, the implementation uses forwarding.

16. Forwarding Mechanism

Two forwarding paths are implemented:

EX/MEM → EX

Used when an ALU result generated by an earlier instruction is immediately required by the next instruction.

This allows the dependent instruction to receive the value without waiting for register write-back.

MEM/WB → EX

Used when the required value is available in the MEM/WB stage.

The implementation records forwarding events and displays them in the pipeline visualization.

Therefore, many RAW hazards are resolved with zero additional stall cycles.

17. Load-Use Hazard

A special case occurs when an instruction immediately uses data loaded by a previous LOAD instruction.

For example:

LOAD → dependent ALU instruction

The loaded value is not available early enough for the dependent EX stage.

Therefore, the implementation inserts:

1-cycle stall

and then forwards the value through:

MEM/WB → EX

This demonstrates the purpose of a hardware interlock.

18. Structural Hazard

The implementation also supports a structural-hazard simulation.

When enabled, the processor assumes a single memory bus is shared between:

- Instruction Fetch
- Memory Access

If the MEM stage is using the memory port while the IF stage attempts to fetch another instruction, a structural conflict occurs.

The implementation then inserts a one-cycle stall.

The optimized configuration disables this contention to represent the use of a split instruction/data memory organization, corresponding to the Harvard-style architecture discussed in the application.

Importantly, the current implementation models this architectural difference; it does not implement a physical cache subsystem.

19. Branch Prediction

The irrigation decision includes a conditional branch:

I9: BGT R6, IRRIGATION_ACTUATE

The implementation evaluates the branch in the EX stage.

Four branch strategies are provided:

1. Always Taken
2. Always Not Taken
3. Dynamic 1-Bit
4. Dynamic 2-Bit Saturating FSM

20. Two-Bit Saturating Counter

The 2-bit predictor uses four states:

State	Meaning	Prediction
00	Strongly Not Taken	Not Taken
01	Weakly Not Taken	Not Taken
10	Weakly Taken	Taken
11	Strongly Taken	Taken

When the actual branch outcome is **Taken**, the state moves toward 11.

When the outcome is **Not Taken**, the state moves toward 00.

The state saturates at both extremes.

This prevents a single unusual branch outcome from immediately reversing the prediction.

21. Branch Prediction Test Sequence

The branch prediction module uses a predefined sequence of **15 simulated irrigation cycles**.

The sequence represents changing environmental conditions such as:

- Dry morning
- Increasing temperature

- Peak heat
- Post-irrigation conditions
- Afternoon conditions
- Evening drying
- Night conditions
- Dawn drying
- Optimal moisture

The sequence is synthetic and is part of the implementation rather than an external agricultural dataset.

22. Branch Prediction Results

For the implemented 15-cycle test sequence, the prediction accuracies are:

Prediction Strategy	Accuracy
Always Taken	46.7%
Always Not Taken	53.3%
Dynamic 1-Bit	80.0%
Dynamic 2-Bit	60.0%

The implementation therefore demonstrates that the 1-bit predictor performs best on this particular hardcoded 15-cycle sequence, while the 2-bit predictor is included to demonstrate saturating-counter FSM behavior.

The report should not claim that the 2-bit predictor achieved the highest accuracy, because the implemented test sequence produces 60.0% for the 2-bit predictor and 80.0% for the 1-bit predictor.

A branch misprediction produces a simulated **2-cycle penalty**, corresponding to flushing the IF and ID stages.

23. Performance Evaluation

The implementation compares two processor configurations.

Unoptimized Configuration

The baseline configuration uses:

- Forwarding: OFF
- Branch prediction: Always Not Taken

- Structural hazard simulation: ON

Optimized Configuration

The optimized configuration uses:

- Forwarding: ON
- Dynamic 2-bit branch prediction
- Structural memory contention disabled

The system compares both configurations using:

- Execution cycles
- CPI
- Throughput
- Latency
- Stall/flush cycles
- Speedup
- Estimated processor energy savings

24. Performance Metrics

The following equations are implemented.

CPI

$$CPI = \frac{\text{Instruction Count}}{\text{Total Execution Cycles}}$$

Throughput

$$\text{Throughput} = \frac{\text{Total Execution Cycles}}{\text{Instruction Count}}$$

Speedup

$$\text{Speedup} = \frac{\text{Optimized Cycles}}{\text{Unoptimized Cycles}}$$

Stall Reduction

$$\text{Stall Reduction} = \frac{\text{Unoptimized Stalls} - \text{Optimized Stalls}}{\text{Unoptimized Stalls}} \times 100$$

Latency

The implementation assumes a **50 MHz processor clock**.

Therefore:

$$\text{Clock Period}=20\text{ns}$$

and:

$$\text{Latency}=\text{TotalCycles}\times 20\text{ns}$$

25. Performance Result for Default Sensor Configuration

For the default input configuration, the implementation produces the following comparison:

Metric	Unoptimized	Optimized
Instructions	11	11
Execution Cycles	35	16
CPI	3.18	1.45
Throughput	0.314 inst/cycle	0.688 inst/cycle
Latency @ 50 MHz	700 ns	320 ns
Stall/Flush Cycles	8	0
Speedup	—	2.19×
Stall Reduction	—	100%
Estimated CPU Energy Saved	—	27.4%

Thus, for the implemented default test case, enabling the architectural optimizations reduces the simulated execution time from 35 cycles to 16 cycles.

The corresponding simulated speedup is:

$$1635=2.19\times$$

26. Result and Validation

The implementation validates the system at three levels.

Level 1 – Agricultural Decision

The sensor values are processed to determine:

- Soil moisture deficit
- Temperature contribution
- Humidity reduction
- Total water demand
- Irrigation status
- Flow rate

For the default input, the output is:

24.65 L/min

with:

Moderate Irrigation Required

Level 2 – Processor Execution

The same processing is represented as an 11-instruction sequence and executed through the simulated:

IF → ID → EX → MEM → WB

pipeline.

The application provides a cycle-by-cycle pipeline visualization.

Level 3 – Architectural Optimization

The implementation compares:

Without optimization

against

Forwarding + branch prediction + reduced memory contention

using measurable performance metrics.

This demonstrates the relationship between processor organization and application-level execution performance.

27. System Output

The final Results module consolidates:

Sensor Input

- Soil moisture
- Temperature
- Humidity
- Target moisture
- Calculated moisture deficit

Processor Execution

- Instructions retired
- Forwarding events
- Hazard information
- Load-use stalls
- Branch outcome

Final Actuation

- Solenoid valve status
- Flow-rate command
- Optimized execution cycles
- CPI
- Speedup

The system also supports exporting the simulation results as:

- **CSV**
- **JSON**

28. Modern Tools Used

The implementation is developed as a web-based interactive simulator using:

Tool / Technology	Purpose
React	User interface and component-based application
TypeScript	Implementation of algorithms and data structures
Vite	Development and build environment
Recharts	Performance and branch prediction charts
Lucide React	Interface icons
Tailwind CSS	User-interface styling
JavaScript Typed Arrays IEEE-754 bit-level representation	

The assignment template specifically asks students to provide evidence of appropriate modern-tool usage and outputs.

Assignment Format - FINAL

29. Advantages of the Proposed System

The implemented system provides the following advantages:

1. Interactive simulation of agricultural sensor inputs.
2. Clear visualization of computer architecture concepts.
3. Demonstration of hardware arithmetic algorithms.
4. Cycle-level pipeline visualization.
5. Explicit RAW hazard detection.
6. Forwarding-path visualization.
7. Load-use stall demonstration.
8. Structural hazard simulation.
9. Branch prediction comparison.
10. Quantitative performance evaluation.
11. CSV and JSON result export.
12. Integration of an agricultural application with processor architecture concepts.

30. Limitations

The current implementation has several limitations.

1. The system uses simulated sensor values rather than physical sensors.
2. No external agricultural dataset is used.
3. The branch prediction evaluation uses a predefined 15-cycle synthetic sequence.
4. The ADC addresses are simulated memory-mapped registers.
5. The solenoid valve is simulated rather than physically controlled.
6. The Harvard-style optimization is represented through the simulator configuration rather than a physical cache implementation.
7. The irrigation model uses fixed coefficients implemented directly in the program.
8. The performance energy-saving value is an estimated simulation metric, not a measurement from physical processor hardware.
9. The current application is intended primarily as an educational computer architecture simulator.

31. Broader Considerations

Sustainability

The application demonstrates how processor-level optimization can reduce simulated execution time and unnecessary stall cycles in an edge-processing workload.

Reducing processing time can potentially be useful for battery-powered agricultural computing nodes. However, the current implementation does not perform physical energy measurements.

Agriculture

The system demonstrates the basic concept of using environmental conditions to determine irrigation requirements rather than applying a fixed irrigation command.

Reliability

The use of multiple environmental parameters allows the decision to consider soil moisture together with temperature and humidity.

Professional Responsibility

Since the current implementation is a simulator, it should not be directly deployed to control real irrigation equipment without additional validation, sensor calibration, safety mechanisms, hardware interfacing, and field testing.

32. Conclusion

The Precision Agriculture using Computer Architecture – Smart Irrigation System successfully integrates an agricultural decision-making problem with fundamental computer architecture concepts.

The implemented system accepts soil moisture, temperature, and humidity values and calculates the required irrigation flow. The processing is represented using an 11-instruction sequence and simulated through a 5-stage RISC pipeline consisting of Instruction Fetch, Instruction Decode, Execute, Memory Access, and Write Back stages.

The project demonstrates Booth's multiplication, Restoring division, IEEE-754 floating-point representation, fixed-point conversion, RAW hazard detection, forwarding, Load-Use interlocks, structural hazard simulation, and branch prediction.

For the default test case, the system calculates an irrigation requirement of 24.65 L/min. The implemented performance comparison shows a reduction from 35 simulated cycles to 16 cycles, corresponding to a $2.19\times$ simulated speedup.

The project therefore demonstrates how processor organization and pipeline optimization can influence the execution of a smart-agriculture workload.

34. Student Reflection

This assignment provided practical understanding of how computer architecture concepts can be applied to an application rather than being treated only as theoretical concepts.

The implementation helped demonstrate the relationship between sensor inputs, arithmetic operations, processor instructions, pipeline stages, hazards, branch prediction, and final application output.

One important learning outcome was understanding that pipelining does not automatically guarantee ideal performance. Data dependencies, Load-Use hazards, structural conflicts, and branch decisions can introduce stalls and flushes.

The implementation also provided practical understanding of how forwarding reduces unnecessary waiting by transferring results directly between pipeline stages.

If additional development time were available, the system could be extended with physical sensor integration, real agricultural datasets, hardware-based validation, adaptive irrigation coefficients, and actual embedded processor measurements.

35. Future Enhancements

The following improvements can be considered in future versions:

1. Connect actual soil-moisture, temperature, and humidity sensors.
2. Interface the system with a physical microcontroller.
3. Control a real irrigation pump or solenoid valve.
4. Replace the synthetic branch sequence with real sensor data.
5. Add historical sensor-data storage.
6. Develop adaptive irrigation coefficients.
7. Implement actual instruction/data cache simulation.
8. Add cache hit/miss analysis.
9. Add more realistic processor power modelling.
10. Test the system using larger agricultural workloads.